

自用XCPC板子(Contest demo)

说明：以下第 1~10 节来自桌面板子.docx（原文），第 11~14 节来自 [ban_zi/](#) 目录，第 15~28 节为补充新板子。最后更新: 2026-07-27

目录

- 1. 线性基
- 2. 分层图最短路（带条件）
- 3. $N \times N$ 矩阵翻转
- 4. 字典树 / Trie
- 5. 组合数计算（阶乘逆元）
- 6. 线性筛 / 埃氏筛 / 质因数分解
- 7. 二分答案
- 8. 三分答案
- 9. 控制小数输出
- 10. 字符串工具库（StringUtils）
- 11. 并查集（Union-Find）
- 12. Dijkstra 最短路
- 13. Floyd 全源最短路
- 14. Kruskal 最小生成树
- 15. 快速幂（qpow）
- 16. 线段树（懒标记）
- 17. 树状数组
- 18. LCA 倍增法
- 19. KMP 字符串匹配
- 20. 字符串哈希
- 21. Manacher 最长回文
- 22. 矩阵快速幂
- 23. 扩展欧几里得（exgcd）
- 24. 欧拉函数（欧拉筛求 φ ）
- 25. 背包 DP
- 26. LIS 最长上升子序列 $O(n \log n)$
- 27. 数位 DP 模板
- 28. SG 函数博弈
- 29. Miller-Rabin 素性测试

1. 线性基

```
const int MAXL = 61;           // 支持  $0 \sim 2^{60}-1$  的数

long long base[MAXL];          // base[i] 表示最高位为 i 的基向量
```

```
bool has_zero; // 标记能否异或出 0

void init() { // 初始化（每次使用前调用）

    memset(base, 0, sizeof(base));

    has_zero = false;

}

// 插入一个数 x

void insert(long long x) {

    for (int i = MAXL - 1; i >= 0; --i) {
        if (!(x >> i & 1)) continue;
        if (base[i] x ^= base[i];
        else {
            base[i] = x;
            return;
        }
    }
    has_zero = true;
}

// 查询最大异或和（非空子集）

long long query_max() {

    long long ans = 0;
    for (int i = MAXL - 1; i >= 0; --i) {
        if ((ans ^ base[i]) > ans) ans ^= base[i];
    }
    return ans;

}

// 查询最小异或和（非空子集；若允许空集，需判断 has_zero）

long long query_min() {

    if (has_zero) return 0;

    for (int i = 0; i < MAXL; ++i) {

        if (base[i]) return base[i];

    }

    return 0; // 理论上不会执行到（除非所有数都是0且has_zero为false，但此时不会插入）

}

// 判断 x 能否被异或得到
```

```
bool check(long long x) {

    for (int i = MAXL - 1; i >= 0; --i) {

        if (x >> i & 1) {

            if (!base[i]) return false;

            x ^= base[i];

        }

    }

    return true;

}

// 合并另一个线性基（另一个基的数组为 other）

void merge_base(const long long other[]) {

    for (int i = 0; i < MAXL; ++i) {

        if (other[i]) insert(other[i]);

    }

}

// 查询第 k 小异或和（k 从 0 开始，0 表示最小，即异或和为 0）

long long kth(long long k) {

    if (has_zero) {

        if (k == 0) return 0;

        --k;    // 跳过 0

    }

    long long tmp[MAXL];

    memcpy(tmp, base, sizeof(base));

    // 高斯消元，化为最简阶梯型

    for (int i = 0; i < MAXL; ++i) {

        if (tmp[i]) {

            for (int j = i - 1; j >= 0; --j) {
```

```

        if ((tmp[i] >> j) & 1) tmp[i] ^= tmp[j];
    }
}

vector<long long> basis;

for (int i = 0; i < MAXL; ++i) {
    if (tmp[i]) basis.push_back(tmp[i]);
}

int sz = basis.size();

if (k >= (1LL << sz)) return -1;

long long ans = 0;

for (int i = 0; i < sz; ++i) {
    if (k >> i & 1) ans ^= basis[i];
}

return ans;
}

```

2. 分层图最短路/带条件

```

typedef pair<int, int> PII;

typedef tuple<ll, int, int> TIII;

const ll INF = 2e18;

void dijkstra(int st, int ed, const vector<vector<PII>>& g, int K) {
    int n = (int)g.size();

    vector<vector<ll>> dist(n, vector<ll>(K + 1, INF));

    priority_queue<TIII, vector<TIII>, greater<TIII>> pq;

    dist[st][0] = 0;
}

```

```
pq.push({0, st, 0});
```

```
while (!pq.empty()) {
```

```
    auto [d, u, layer] = pq.top(); pq.pop();
```

```
    if (d != dist[u][layer]) continue; //旧数据去除
```

//中间自由发挥，主要写限制条件循环和多层的状态转移循环，在tuple里写的一般是（代价，点集，状态）

//dist的定义永远比pq的定义少一维，因为dist[][]..就是一个维度

//状态是多样的，也可以是有因果关系的，自行推导关系，可以思考状压，位运算等往往是求关系的难点

```
    for (auto [v, w] : g[u]) {
```

```
        ll new_d = dist[u][layer] + w;
```

```
        if (dist[v][layer] > new_d) {
```

```
            dist[v][layer] = new_d;
```

```
            pq.push({dist[v][layer], v, layer});
```

```
        }
```

```
    }
```

```
    if (layer < K) {
```

```
        for (auto [v, w] : g[u]) {
```

```
            ll new_d = dist[u][layer];
```

```
            if (dist[v][layer + 1] > new_d) {
```

```
                dist[v][layer + 1] = new_d;
```

```
                pq.push({dist[v][layer + 1], v, layer + 1});
```

```
            }
```

```
        }
```

```
    }
```

```
}
```

```
ll ans = INF;
```

```
for (int j = 0; j <= K; ++j) ans = min(ans, dist[ed][j]);
```

```
    if (ans == INF) cout << -1 << '\n';

    else cout << ans << '\n';

}

void solve() {

    int n, m, k;

    cin >> n >> m >> k;

    vector<vector<PII>> g(n+1);

    for (int i = 0; i < m; ++i) {

        int u, v, w;

        cin >> u >> v >> w;

        g[u].push_back({v, w});

        g[v].push_back({u, w});

    }

    dijkstra(1, n, g, k);

}
```

3. N*N矩阵翻转

```
void rotate90(vector<vector<char>>& matrix) {

    int n = matrix.size();

    for (int i = 0; i < n; ++i) {

        for (int j = i + 1; j < n; ++j) {

            swap(matrix[i][j], matrix[j][i]);

        }

    }

    for (int i = 0; i < n; ++i) {

        reverse(matrix[i].begin(), matrix[i].end());

    }

}
```

```

    }

}

void rotate180(vector<vector<char>>& matrix) {

    int n = matrix.size();

    for(int i = 0; i < n; i++){

        for(int j = 0; j < (n>>1); j++){

            swap(matrix[j][i],matrix[n-j-1][i]);

        }

    }

    for (int i = 0; i < n; ++i) {

        reverse(matrix[i].begin(), matrix[i].end());

    }

}

}

```

4. 字典树

```

// 【重要】N 开多大？

// 等于「所有插入字符串的总长度 + 5」。

// 竞赛一般会给总长度上限，如 1e5 或 1e6。

// 若上限 1e5，开 100010；若 1e6，开 1000010。

// 注意内存：1e6 * 26 * 4 = 104 MB，看清题目内存限制（一般 256MB 够）。

const int N = 1000010;

int son[N][26]; // 子节点

int cnt[N];     // 以该节点结尾的单词数量

int idx;        // 当前用到哪个节点（0 既是根也是空）

// 插入

```

```
void insert(char *str) { // 竞赛用 char* 速度略快于 string

    int p = 0;

    for (int i = 0; str[i]; i++) {

        int u = str[i] - 'a';

        if (!son[p][u]) son[p][u] = ++ idx;

        p = son[p][u];

    }

    cnt[p]++;

}

// 查询单词出现次数

int query(char *str) {

    int p = 0;

    for (int i = 0; str[i]; i++) {

        int u = str[i] - 'a';

        if (!son[p][u]) return 0;

        p = son[p][u];

    }

    return cnt[p];

}

// 【竞赛高频】多组测试数据时，清空字典树（比 memset 整个二维数组快得多）

void clear_trie() {

    for (int i = 0; i <= idx; i++) {

        // 只清空用过的节点，0 号根节点也会被清

        memset(son[i], 0, sizeof(son[i]));

        cnt[i] = 0;

    }

    idx = 0; // 别忘了重置编号

}
```

```
}
```

5. 组合数计算（预处理阶乘 + 逆元， $O(1)$ 查询）

```
const int MOD = 1'000'000'007;

const int MAXN = 1'000'000;

ll fac[MAXN + 1], invfac[MAXN + 1];

ll qpow(ll a, ll b) {
    ll res = 1;
    while (b) {
        if (b & 1) res = res * a % MOD;
        a = a * a % MOD;
        b >>= 1;
    }
    return res;
}

void init_comb(int n) {           // 预处理到 n
    fac[0] = 1;
    for (int i = 1; i <= n; i++) fac[i] = fac[i - 1] * i % MOD;
    invfac[n] = qpow(fac[n], MOD - 2);
    for (int i = n - 1; i >= 0; i--) invfac[i] = invfac[i + 1] * (i + 1) % MOD;
}

ll C(int n, int k) {
    if (k < 0 || k > n) return 0;
    return fac[n] * invfac[k] % MOD * invfac[n - k] % MOD;
}
```

6. 线性筛 / 埃氏筛 / 质因数分解

// 线性筛 - 传入上限 n ($n \geq 2$)，返回 $[2, n]$ 中的所有质数

```
vector<int> linear_sieve(int n) {  
    vector<bool> is_prime(n + 1, true);  
    vector<int> primes;  
    for(int i = 2; i <= n; i++) {  
        if(is_prime[i]) {  
            primes.emplace_back(i);  
        }  
        for(auto p : primes) {  
            if(i * p > n) break;  
            is_prime[i * p] = false;  
            if(i % p == 0) break;  
        }  
    }  
    return primes;  
}
```

// 埃氏筛 - 传入上限 n ($n \geq 2$)，返回 $[2, n]$ 中的所有质数

```
vector<int> eratosthenes_sieve(int n) {  
    vector<bool> is_prime(n + 1, true);  
    vector<int> primes;  
    for(int i = 2; i <= n; i++) {  
        if(is_prime[i]) {  
            primes.emplace_back(i);  
            // 从  $i*i$  开始筛（更小的倍数已被更小的质数筛过）  
            if((long long)i * i <= n) {  
                is_prime[i * i] = false;  
                for(int j = i * i; j <= n; j += i) {  
                    is_prime[j] = false;  
                }  
            }  
        }  
    }  
    return primes;  
}
```

```
        for(int j = i * i; j <= n; j += i) {

            is_prime[j] = false;

        }

    }

}

return primes;

}

// 质因数分解 - 需要传入已经筛好的质数表

vector<int> factorize(int x, const vector<int>& primes) {

    vector<int> factors;

    for(auto p : primes) {

        if((long long)p * p > x) break;

        while(x % p == 0) {

            factors.emplace_back(p);

            x /= p;

        }

    }

    if(x > 1) factors.emplace_back(x);

    return factors;

}
```

7. 二分答案

```
bool check(int x) {

    // TODO: 判断 x 是否可行

    // 例如: 能否使任意两牛距离 >= x

}
```

```
    return true; // 或 false
}

int solve_max_min(int l, int r) { // l 为最小可能, r 为最大可能

    while (l < r) {

        int mid = (l + r + 1) >> 1; // 注意 +1, 防止死循环

        if (check(mid)) {

            l = mid;          // mid 可行, 尝试更大的

        } else {

            r = mid - 1;      // mid 不可行, 减小范围

        }

    }

    return l;

}

//动 r (右边界) → mid = l + ((r - l) >> 1)

动 l (左边界) → mid = l + ((r - l + 1) >> 1)
```

8. 三分答案

```
// 以"求最大值"为例 (如  $f(x) = -x^2 + 1$ )

double f(double x) {

    // TODO: 返回函数值

    return x * x;

}

double solve(double l, double r) {

    // 精度控制: 比题目要求多两位

    for (int i = 0; i < 100; i++) { // 固定100次, 绝对安全, 无需eps

        double m1 = l + (r - l) / 3;
```

```
double m2 = r - (r - 1) / 3;

if (f(m1) < f(m2)) {

    l = m1; // 最大值在右边（若求最小值则方向相反）

} else {

    r = m2;

}

}

return (l + r) / 2; // 或返回 f((l+r)/2)

}
```

9. 控制小数输出

```
#include <iomanip> // 必须包含

#include <iostream>

int main() {

    double num = 3.1415926;

    // 1. 保留 3 位小数（最常用）

    cout << fixed << setprecision(3) << num << endl; // 输出 3.142

    // 2. 如果想取消 fixed，恢复显示有效数字

    cout << setprecision(3) << num << endl;           // 输出 3.14（3位有效数字）

    // 3. 输出百分比或去掉末尾无用的0（取消 fixed 即可）

    cout << setprecision(2) << 3.1000 << endl;       // 输出 3.1

    return 0;

}
```

10. 字符串工具库（StringUtils）

```
#include <bits/stdc++.h>

using namespace std;

namespace StringUtils {

    // ===== 1. 大小写转换 =====

    string toLower(string s) {

        for (char& c : s) c = tolower(c);

        return s;

    }

    string toUpper(string s) {

        for (char& c : s) c = toupper(c);

        return s;

    }

    // ===== 2. 去除首尾指定字符（默认空格） =====

    string ltrim(string s, char ch = ' ') {

        size_t pos = s.find_first_not_of(ch);

        if (pos == string::npos) return "";

        return s.substr(pos);

    }

    string rtrim(string s, char ch = ' ') {

        size_t pos = s.find_last_not_of(ch);

        if (pos == string::npos) return "";

        return s.substr(0, pos + 1);

    }

    string trim(string s, char ch = ' ') {

        return rtrim(ltrim(s, ch), ch);

    }

}
```

```
// ===== 3. 字符串分割 =====

vector<string> split(const string& s, char delim) {

    vector<string> res;

    string cur;

    for (char c : s) {

        if (c == delim) {

            if (!cur.empty()) res.push_back(cur);

            cur.clear();

        } else {

            cur.push_back(c);

        }

    }

    if (!cur.empty()) res.push_back(cur);

    return res;

}

// 按字符串分割（更通用）

vector<string> split(const string& s, const string& delim) {

    vector<string> res;

    size_t start = 0, end;

    while ((end = s.find(delim, start)) != string::npos) {

        string sub = s.substr(start, end - start);

        if (!sub.empty()) res.push_back(sub);

        start = end + delim.size();

    }

    string last = s.substr(start);

    if (!last.empty()) res.push_back(last);

    return res;

}
```

```
}

// ===== 4. 判断是否回文 =====

bool isPalindrome(const string& s) {

    int l = 0, r = (int)s.size() - 1;

    while (l < r) {

        if (s[l++] != s[r--]) return false;

    }

    return true;

}

// ===== 5. 检查前缀/后缀 =====

bool startsWith(const string& s, const string& prefix) {

    if (prefix.size() > s.size()) return false;

    return s.compare(0, prefix.size(), prefix) == 0;

}

bool endsWith(const string& s, const string& suffix) {

    if (suffix.size() > s.size()) return false;

    return s.compare(s.size() - suffix.size(), suffix.size(), suffix) == 0;

}

// ===== 6. 整数与字符串互转（支持 __int128） =====

string toString(long long x) {

    return to_string(x);

}

long long toLL(const string& s) {

    return stoll(s);

}

// 如果支持 __int128 (GCC)

string i128ToString(__int128 x) {
```

```
    if (x == 0) return "0";

    bool neg = x < 0;

    if (neg) x = -x;

    string res;

    while (x > 0) {

        res.push_back('0' + x % 10);

        x /= 10;

    }

    if (neg) res.push_back('-');

    reverse(res.begin(), res.end());

    return res;

}

__int128 toI128(const string& s) {

    __int128 res = 0;

    bool neg = false;

    int i = 0;

    if (s[i] == '-') { neg = true; i++; }

    for (; i < (int)s.size(); ++i) {

        res = res * 10 + (s[i] - '0');

    }

    return neg ? -res : res;

}

// ===== 7. 大数加法（非负） =====

string addBig(const string& a, const string& b) {

    int i = a.size() - 1, j = b.size() - 1, carry = 0;

    string res;

    while (i >= 0 || j >= 0 || carry) {
```

```
        int sum = carry;

        if (i >= 0) sum += a[i--] - '0';

        if (j >= 0) sum += b[j--] - '0';

        res.push_back('0' + sum % 10);

        carry = sum / 10;
    }

    reverse(res.begin(), res.end());

    return res;
}

// ===== 8. 大数减法 (a >= b > 0) =====

string subBig(const string& a, const string& b) {

    int i = a.size() - 1, j = b.size() - 1, borrow = 0;

    string res;

    while (i >= 0 || j >= 0) {

        int diff = borrow;

        if (i >= 0) diff += a[i--] - '0';

        if (j >= 0) diff -= b[j--] - '0';

        if (diff < 0) {

            diff += 10;

            borrow = -1;

        } else {

            borrow = 0;

        }

        res.push_back('0' + diff);

    }

    // 去掉前导零

    size_t pos = res.find_last_not_of('0');
```

```
        if (pos == string::npos) return "0";

        res = res.substr(0, pos + 1);

        reverse(res.begin(), res.end());

        return res;
    }

    // ===== 9. 大数乘单个整数 =====

    string mulBig(const string& a, int b) {

        if (b == 0) return "0";

        int carry = 0;

        string res;

        for (int i = a.size() - 1; i >= 0; --i) {

            int prod = (a[i] - '0') * b + carry;

            res.push_back('0' + prod % 10);

            carry = prod / 10;

        }

        while (carry) {

            res.push_back('0' + carry % 10);

            carry /= 10;

        }

        reverse(res.begin(), res.end());

        return res;

    }

    // ===== 10. 大数比较（不带符号） =====

    int cmpBig(const string& a, const string& b) {

        if (a.size() != b.size()) return a.size() < b.size() ? -1 : 1;

        return a.compare(b); // 相等返回0，小于返回负，大于返回正

    }
```

```
// ===== 11. 字符串重复 =====

string repeat(const string& s, int times) {

    string res;

    res.reserve(s.size() * times);

    while (times--) res += s;

    return res;

}

// ===== 12. 替换所有子串 =====

string replaceAll(string s, const string& from, const string& to) {

    size_t pos = 0;

    while ((pos = s.find(from, pos)) != string::npos) {

        s.replace(pos, from.size(), to);

        pos += to.size();

    }

    return s;

}

} // namespace StringUtils

int main() {

    using namespace StringUtils;

    string s = "    Hello, World!    ";

    cout << trim(s) << '\n';           // "Hello, World!"

    vector<string> parts = split(s, ", ");

    for (auto& p : parts) cout << p << '\n'; // "    Hello" 和 "World!    " (含空格)

    cout << isPalindrome("abba") << '\n'; // 1

    string a = "12345678901234567890";

    string b = "98765432109876543210";

    cout << addBig(a, b) << '\n';      // 111111111011111111100
```

```
cout << subBig(b, a) << '\n';    // 86419753208641975320

cout << toUpper("hello") << '\n'; // HELLO

cout << repeat("ab", 3) << '\n';  // ababab

cout << replaceAll("hello world", "world", "C++") << '\n'; // hello C++

return 0;

}
```

11. 并查集 (Union-Find)

```
// 来自 ban_zi/union_ck.cpp

const int MAXN = 100010;
int parent[MAXN], rnk[MAXN]; // rnk 用于按秩合并

void init(int n) {
    for (int i = 0; i < n; ++i) {
        parent[i] = i;
        rnk[i] = 0;
    }
}

int find(int x) {
    if (parent[x] != x) parent[x] = find(parent[x]);
    return parent[x];
}

bool unite(int x, int y) {
    int rx = find(x), ry = find(y);
    if (rx == ry) return false;
    if (rnk[rx] < rnk[ry]) parent[rx] = ry;
    else if (rnk[rx] > rnk[ry]) parent[ry] = rx;
    else {
        parent[ry] = rx;
        rnk[rx]++;
    }
    return true;
}
```

12. Dijkstra 最短路

```
// 来自 ban_zi/dijkstra.cpp
```

```

using ll = long long;
using PII = pair<int, int>; // {距离, 节点}
const ll INF = 0x3f3f3f3f3f3f3f3f;

vector<ll> dijkstra(int n, vector<vector<PII>> &g, int s) {
    vector<ll> dist(n, INF);
    vector<bool> vis(n, false);
    priority_queue<PII, vector<PII>, greater<PII>> pq;

    dist[s] = 0;
    pq.push({0, s});

    while (!pq.empty()) {
        auto [d, u] = pq.top();
        pq.pop();
        if (vis[u]) continue;
        vis[u] = true;
        for (auto &[v, w] : g[u]) {
            if (dist[v] > d + w) {
                dist[v] = d + w;
                pq.push({dist[v], v});
            }
        }
    }
    return dist;
}

```

13. Floyd 全源最短路

```

// 来自 ban_zi/floyd.cpp

const int max_node = 300;
vector<vector<int>> mp(max_node+1, vector<int>(max_node+1, INT32_MAX));

void floyd(){
    for(int k = 1; k <= max_node; k++){
        for(int i = 1; i <= max_node; i++){
            for(int j = 1; j <= max_node; j++){
                if(mp[i][k] != INT32_MAX && mp[k][j] != INT32_MAX
                    && (mp[i][k] + mp[k][j] < mp[i][j]))
                    mp[i][j] = mp[i][k] + mp[k][j];
            }
        }
    }
}

```

14. Kruskal 最小生成树

```
// 来自 ban_zi/kruskal.cpp

struct Edge {
    int u, v, w;
};

int n, m;
vector<Edge> edges;
int parent[100010];

int find(int x) {
    if (parent[x] != x) parent[x] = find(parent[x]);
    return parent[x];
}

void unify(int a, int b) {
    int ra = find(a), rb = find(b);
    if (ra != rb) parent[rb] = ra;
}

pair<int, vector<Edge>> kruskal() {
    sort(edges.begin(), edges.end(), [](Edge a, Edge b) { return a.w < b.w; });
    for (int i = 1; i <= n; i++) parent[i] = i;
    int total_weight = 0, edge_cnt = 0;
    vector<Edge> mst_edges;
    for (auto &e : edges) {
        if (find(e.u) != find(e.v)) {
            unify(e.u, e.v);
            total_weight += e.w;
            mst_edges.push_back(e);
            edge_cnt++;
            if (edge_cnt == n - 1) break;
        }
    }
    if (edge_cnt == n - 1) return {total_weight, mst_edges};
    else return {-1, {}};
}
```

15. 快速幂 (qpow)

```
// 快速幂（模意义下）
ll qpow(ll a, ll b, ll mod) {
    ll res = 1 % mod;
    while (b) {
        if (b & 1) res = res * a % mod;
        a = a * a % mod;
        b >>= 1;
    }
    return res;
}
```

```

}

// 快速乘（防溢出）
ll qmul(ll a, ll b, ll mod) {
    ll res = 0;
    while (b) {
        if (b & 1) res = (res + a) % mod;
        a = (a + a) % mod;
        b >>= 1;
    }
    return res;
}

```

16. 线段树（懒标记）

```

const int N = 100010;
ll a[N];           // 原数组（1-indexed）

struct SegTree {
    ll sum[N << 2];
    ll lazy[N << 2];    // 懒标记（加法）

    void push_up(int rt) {
        sum[rt] = sum[rt << 1] + sum[rt << 1 | 1];
    }

    void push_down(int rt, int l, int r) {
        if (lazy[rt]) {
            int mid = (l + r) >> 1;
            int lc = rt << 1, rc = rt << 1 | 1;
            lazy[lc] += lazy[rt];
            sum[lc] += lazy[rt] * (mid - l + 1);
            lazy[rc] += lazy[rt];
            sum[rc] += lazy[rt] * (r - mid);
            lazy[rt] = 0;
        }
    }

    void build(int rt, int l, int r) {
        if (l == r) { sum[rt] = a[l]; return; }
        int mid = (l + r) >> 1;
        build(rt << 1, l, mid);
        build(rt << 1 | 1, mid + 1, r);
        push_up(rt);
    }

    // 区间加
    void update(int rt, int l, int r, int L, int R, ll val) {
        if (L <= l && r <= R) {
            sum[rt] += val * (r - l + 1);

```

```

        lazy[rt] += val;
        return;
    }
    push_down(rt, l, r);
    int mid = (l + r) >> 1;
    if (L <= mid) update(rt << 1, l, mid, L, R, val);
    if (R > mid) update(rt << 1 | 1, mid + 1, r, L, R, val);
    push_up(rt);
}

// 区间查询
ll query(int rt, int l, int r, int L, int R) {
    if (L <= l && r <= R) return sum[rt];
    push_down(rt, l, r);
    int mid = (l + r) >> 1;
    ll res = 0;
    if (L <= mid) res += query(rt << 1, l, mid, L, R);
    if (R > mid) res += query(rt << 1 | 1, mid + 1, r, L, R);
    return res;
}
} seg;

```

17. 树状数组

```

int n;
vector<ll> bit;

void init(int _n) {
    n = _n;
    bit.assign(n + 1, 0);
}

// 单点加
void add(int idx, ll delta) {
    while (idx <= n) {
        bit[idx] += delta;
        idx += idx & -idx;
    }
}

// 前缀和 [1, idx]
ll sum(int idx) {
    ll res = 0;
    while (idx > 0) {
        res += bit[idx];
        idx -= idx & -idx;
    }
    return res;
}

```

```
// 区间和 [l, r]
ll range_sum(int l, int r) {
    return sum(r) - sum(l - 1);
}

// 树上二分：找最小的 idx 使得前缀和 >= target（需要所有数非负）
int kth(ll target) {
    int idx = 0;
    int bit_mask = 1 << (__lg(n));
    for (; bit_mask; bit_mask >>= 1) {
        int nxt = idx + bit_mask;
        if (nxt <= n && bit[nxt] < target) {
            target -= bit[nxt];
            idx = nxt;
        }
    }
    return idx + 1;
}
```

18. LCA 倍增法

```
const int N = 100010;
const int LOG = 17; // 2^17 = 131072 > 1e5

vector<int> g[N];
int up[N][LOG + 1];
int depth[N];

void dfs_lca(int u, int p) {
    up[u][0] = p;
    depth[u] = depth[p] + 1;
    for (int k = 1; k <= LOG; ++k) {
        up[u][k] = up[up[u][k - 1]][k - 1];
    }
    for (int v : g[u]) {
        if (v != p) dfs_lca(v, u);
    }
}

int lca(int u, int v) {
    if (depth[u] < depth[v]) swap(u, v);
    int diff = depth[u] - depth[v];
    for (int k = 0; k <= LOG; ++k) {
        if (diff >> k & 1) u = up[u][k];
    }
    if (u == v) return u;
    for (int k = LOG; k >= 0; --k) {
        if (up[u][k] != up[v][k]) {
            u = up[u][k];
            v = up[v][k];
        }
    }
    return up[u][0];
}
```

```

    }
}
return up[u][0];
}

int dist(int u, int v) {
    int l = lca(u, v);
    return depth[u] + depth[v] - 2 * depth[l];
}

```

19. KMP 字符串匹配

```

// 计算 next 数组
vector<int> get_next(const string &pat) {
    int m = pat.size();
    vector<int> nxt(m);
    nxt[0] = 0;
    for (int i = 1, j = 0; i < m; ++i) {
        while (j > 0 && pat[i] != pat[j]) j = nxt[j - 1];
        if (pat[i] == pat[j]) ++j;
        nxt[i] = j;
    }
    return nxt;
}

// KMP 匹配, 返回所有匹配起始位置
vector<int> kmp(const string &txt, const string &pat) {
    int n = txt.size(), m = pat.size();
    if (m == 0) return {};
    vector<int> nxt = get_next(pat);
    vector<int> res;
    for (int i = 0, j = 0; i < n; ++i) {
        while (j > 0 && txt[i] != pat[j]) j = nxt[j - 1];
        if (txt[i] == pat[j]) ++j;
        if (j == m) {
            res.push_back(i - m + 1);
            j = nxt[j - 1];
        }
    }
    return res;
}

```

20. 字符串哈希

```

// 双哈希（避免碰撞）
struct Hash {
    static const int MOD1 = 1'000'000'007;

```

```

static const int MOD2 = 1'000'000'009;
static const int BASE = 131;

vector<long long> h1, h2, p1, p2;

Hash(const string &s) {
    int n = s.size();
    h1.resize(n + 1);
    h2.resize(n + 1);
    p1.resize(n + 1);
    p2.resize(n + 1);
    p1[0] = p2[0] = 1;
    for (int i = 1; i <= n; ++i) {
        int c = s[i - 1] - 'a' + 1;
        h1[i] = (h1[i - 1] * BASE + c) % MOD1;
        h2[i] = (h2[i - 1] * BASE + c) % MOD2;
        p1[i] = p1[i - 1] * BASE % MOD1;
        p2[i] = p2[i - 1] * BASE % MOD2;
    }
}

// 返回子串 [l, r) 的哈希值 (0-indexed)
pair<long long, long long> get(int l, int r) {
    long long v1 = (h1[r] - h1[l] * p1[r - l] % MOD1 + MOD1) % MOD1;
    long long v2 = (h2[r] - h2[l] * p2[r - l] % MOD2 + MOD2) % MOD2;
    return {v1, v2};
}
};

```

21. Manacher 最长回文

```

// 返回以每个位置为中心的最长回文半径 (含中心)
vector<int> manacher(const string &s) {
    string t = "#";
    for (char c : s) { t += c; t += '#'; }
    int n = t.size();
    vector<int> d(n);
    int l = 0, r = -1;
    for (int i = 0; i < n; ++i) {
        int k = (i > r) ? 1 : min(d[l + r - i], r - i + 1);
        while (i - k >= 0 && i + k < n && t[i - k] == t[i + k]) ++k;
        d[i] = k;
        if (i + k - 1 > r) {
            l = i - k + 1;
            r = i + k - 1;
        }
    }
    return d;
}

```

```

}
// 最长回文子串长度 = max(d) - 1

```

22. 矩阵快速幂

```

struct Mat {
    int n;
    vector<vector<ll>> a;
    Mat(int _n, int val = 0) : n(_n), a(_n, vector<ll>(_n, 0)) {
        if (val == 1) {
            for (int i = 0; i < n; ++i) a[i][i] = 1;
        }
    }

    Mat operator*(const Mat &rhs) const {
        Mat res(n);
        for (int i = 0; i < n; ++i) {
            for (int k = 0; k < n; ++k) {
                if (a[i][k] == 0) continue;
                for (int j = 0; j < n; ++j) {
                    res.a[i][j] = (res.a[i][j] + a[i][k] * rhs.a[k][j]) % MOD;
                }
            }
        }
        return res;
    }
};

Mat mat_pow(Mat base, ll exp) {
    Mat res(base.n, 1);
    while (exp) {
        if (exp & 1) res = res * base;
        base = base * base;
        exp >>= 1;
    }
    return res;
}

```

23. 扩展欧几里得 (exgcd)

```

// 解 ax + by = gcd(a, b)
ll exgcd(ll a, ll b, ll &x, ll &y) {
    if (b == 0) { x = 1; y = 0; return a; }
    ll g = exgcd(b, a % b, y, x);
    y -= a / b * x;
    return g;
}

```

```
// 求逆元
ll inv(ll a, ll mod) {
    ll x, y;
    exgcd(a, mod, x, y);
    return (x % mod + mod) % mod;
}

// 解线性同余方程  $ax \equiv b \pmod{m}$ , 无解返回 -1
ll solve_congruence(ll a, ll b, ll m) {
    ll x, y;
    ll g = exgcd(a, m, x, y);
    if (b % g != 0) return -1;
    x = (x * (b / g)) % m;
    if (x < 0) x += m;
    return x;
}
```

24. 欧拉函数（欧拉筛求 φ ）

```
// 线性筛同时求欧拉函数  $\phi$ 
vector<int> get_phi(int n) {
    vector<int> phi(n + 1);
    vector<int> primes;
    vector<bool> is_prime(n + 1, true);
    phi[1] = 1;
    for (int i = 2; i <= n; ++i) {
        if (is_prime[i]) {
            primes.push_back(i);
            phi[i] = i - 1;
        }
        for (int p : primes) {
            if (i * p > n) break;
            is_prime[i * p] = false;
            if (i % p == 0) {
                phi[i * p] = phi[i] * p;
                break;
            } else {
                phi[i * p] = phi[i] * (p - 1);
            }
        }
    }
    return phi;
}

// 单个数求欧拉函数  $O(\sqrt{n})$ 
ll get_phi_single(ll n) {
    ll res = n;
    for (ll i = 2; i * i <= n; ++i) {
        if (n % i == 0) {
```

```

        res = res / i * (i - 1);
        while (n % i == 0) n /= i;
    }
}
if (n > 1) res = res / n * (n - 1);
return res;
}

```

25. 背包 DP

```

// ===== 0/1 背包 =====
vector<int> zero_one_knap(const vector<int> &weights,
                        const vector<int> &values, int capacity) {
    vector<int> dp(capacity + 1, 0);
    int n = weights.size();
    for (int i = 0; i < n; ++i) {
        for (int j = capacity; j >= weights[i]; --j) {
            dp[j] = max(dp[j], dp[j - weights[i]] + values[i]);
        }
    }
    return dp;
}

// ===== 完全背包 =====
vector<int> complete_knap(const vector<int> &weights,
                        const vector<int> &values, int capacity) {
    vector<int> dp(capacity + 1, 0);
    int n = weights.size();
    for (int i = 0; i < n; ++i) {
        for (int j = weights[i]; j <= capacity; ++j) {
            dp[j] = max(dp[j], dp[j - weights[i]] + values[i]);
        }
    }
    return dp;
}

// ===== 多重背包（二进制优化） =====
vector<int> multiple_knap(const vector<int> &weights,
                        const vector<int> &values,
                        const vector<int> &counts, int capacity) {
    vector<int> dp(capacity + 1, 0);
    int n = weights.size();
    for (int i = 0; i < n; ++i) {
        int w = weights[i], v = values[i], c = counts[i];
        int k = 1;
        while (k <= c) {
            int nw = w * k, nv = v * k;
            for (int j = capacity; j >= nw; --j) {
                dp[j] = max(dp[j], dp[j - nw] + nv);
            }
        }
    }
}

```

```

        c -= k; k <= 1;
    }
    if (c > 0) {
        int nw = w * c, nv = v * c;
        for (int j = capacity; j >= nw; --j) {
            dp[j] = max(dp[j], dp[j - nw] + nv);
        }
    }
}
return dp;
}

```

26. LIS 最长上升子序列 $O(n \log n)$

```

// 最长上升子序列（严格上升）
int LIS(const vector<int> &a) {
    vector<int> tail;
    for (int x : a) {
        auto it = lower_bound(tail.begin(), tail.end(), x);
        if (it == tail.end()) tail.push_back(x);
        else *it = x;
    }
    return tail.size();
}

// 最长非降子序列（允许相等）
int LNDS(const vector<int> &a) {
    vector<int> tail;
    for (int x : a) {
        auto it = upper_bound(tail.begin(), tail.end(), x);
        if (it == tail.end()) tail.push_back(x);
        else *it = x;
    }
    return tail.size();
}

```

27. 数位 DP 模板

```

// 以统计  $[0, n]$  中不含 49 的数字个数为例
ll dp[20][2][2];
int digits[20];

ll dfs(int pos, bool pre_is_4, bool limit, bool lead) {
    if (pos < 0) return 1;
    if (!limit && !lead && dp[pos][pre_is_4][limit] != -1)
        return dp[pos][pre_is_4][limit];
}

```

```

    int up = limit ? digits[pos] : 9;
    ll res = 0;
    for (int d = 0; d <= up; ++d) {
        if (pre_is_4 && d == 9) continue;
        res += dfs(pos - 1, d == 4, limit && d == up, lead && d == 0);
    }
    if (!limit && !lead) dp[pos][pre_is_4][limit] = res;
    return res;
}

ll solve(ll x) {
    if (x < 0) return 0;
    int len = 0;
    while (x) { digits[len++] = x % 10; x /= 10; }
    memset(dp, -1, sizeof(dp));
    return dfs(len - 1, false, true, true);
}

```

28. SG 函数博弈

```

const int MAXN = 1000;
int sg[MAXN];
bool vis[MAXN];

void calc_sg(int n, const vector<int> &moves) {
    sg[0] = 0;
    for (int i = 1; i <= n; ++i) {
        memset(vis, 0, sizeof(vis));
        for (int m : moves) {
            if (i >= m) vis[sg[i - m]] = true;
        }
        for (int j = 0;; ++j) {
            if (!vis[j]) { sg[i] = j; break; }
        }
    }
}

// Nim 游戏: 异或和为 0 则先手必败
bool nim(const vector<int> &piles) {
    int xorsum = 0;
    for (int x : piles) xorsum ^= x;
    return xorsum != 0;
}

```

29. Miller-Rabin 素性测试

```
ll qpow_mod(ll a, ll b, ll mod) {
    ll res = 1;
    while (b) {
        if (b & 1) res = (__int128)res * a % mod;
        a = (__int128)a * a % mod;
        b >>= 1;
    }
    return res;
}

bool is_prime(ll n) {
    if (n < 2) return false;
    if (n == 2 || n == 3) return true;
    if (n % 2 == 0) return false;

    ll d = n - 1;
    int r = 0;
    while (d % 2 == 0) { d /= 2; r++; }

    vector<ll> bases = {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37};
    for (ll a : bases) {
        if (a >= n) break;
        ll x = qpow_mod(a, d, n);
        if (x == 1 || x == n - 1) continue;
        bool ok = false;
        for (int i = 1; i < r; ++i) {
            x = (__int128)x * x % n;
            if (x == n - 1) { ok = true; break; }
        }
        if (!ok) return false;
    }
    return true;
}
```

说明：第 1~10 节来自你桌面的 [板子.docx](#)（原文），第 11~14 节来自 [ban_zi/](#) 目录，第 15~29 节为补充的新板子。使用代码时请根据需要调整常量（如 MAXN、MOD 等）。